



CS2013 — Programación III

Proyecto 1

Circuito de Escape

Juego de navegación por turnos con C++20

Rubén Rivas Medina
Setiembre de 2026

Propósito

Diseñar un motor de navegación reproducible y separado de su interfaz para integrar tipos abstractos, templates, concepts, polimorfismo dinámico, iteradores, algoritmos y contenedores de la biblioteca estándar.

Contenido

1. Descripción general	2
2. Duración y modalidad	2
3. Objetivos	2
4. Escenario del juego	2
5. Alcance funcional mínimo	3
5.1 Tablero	3
5.2 Posición y dirección	3
5.3 Agente	3
5.4 Movimiento	3
5.5 Tipos de terreno e interacciones	4
5.6 Controladores	5
5.7 Contratos públicos mínimos	5
5.8 Interfaz mínima de consola	9
6. Requisitos de diseño y programación	12
6.1 Plantillas de funciones	12
6.2 Plantillas de clases	12
6.3 Especialización	12
6.4 Plantillas variádicos	13
6.5 Concepts y polimorfismo dinámico	13
6.6 Biblioteca estándar	14
7. Preparación para el segundo proyecto	14
Estado observable	14
Resultado de un paso	14
8. Pruebas mínimas	15
9. Restricciones	15
10. Organización y formato de entrega	16
10.1 Estructura mínima del repositorio	16
10.2 Contenido obligatorio	16
10.3 Entrega mediante GitHub	16
10.4 Flujo de branches y pull requests	17
10.5 Formación del grupo y división del trabajo	17
11. Plan de trabajo sugerido	18
Semana 1: tablero y estado	18
Semana 2: entorno	18
Semana 3: extensibilidad	18
Semana 4: integración	18
12. Posibles ventajas para la segunda etapa	18
13. Riesgos de alcance	18
14. Decisiones pendientes	19

1. Descripción general

Se desarrollará en C++20 el núcleo de un juego de navegación por turnos. Un agente debe recorrer una cuadrícula, evitar obstáculos, administrar su energía, recoger recursos y alcanzar una salida.

El proyecto se concentrará en el motor de simulación y no en su presentación. No se requiere interfaz gráfica, animaciones, conexión en red ni persistencia en una base de datos. El tablero se mostrará mediante emojis en una consola con soporte UTF-8 y ofrecerá una representación ASCII alternativa.

El motor deberá permitir ejecutar una partida de forma interactiva y también de forma automática, sin depender de `std::cin` o `std::cout`. Esta separación será importante porque, en un proyecto posterior, se incorporará una estrategia simple de inteligencia artificial y se podrá experimentar con Q-learning tabular u otro algoritmo introductorio de reinforcement learning.

2. Duración y modalidad

- Duración: 4 semanas.
- Lenguaje: C++20.
- Interfaz: consola.
- Construcción: CMake.
- Pruebas: framework indicado en clase o programas de prueba con `assert`.
- Modalidad de trabajo: grupal, con un máximo de cinco integrantes.
- Entrega: repositorio GitHub por grupo, con historial de contribuciones visible.

3. Objetivos

El proyecto permitirá practicar:

- diseño de tipos de datos abstractos e invariantes;
- copia, movimiento y administración de recursos mediante RAI;
- sobrecarga de funciones y operadores cuando sea apropiado;
- herencia y polimorfismo dinámico;
- templates de funciones y clases;
- parámetros de tipo y parámetros no-tipo;
- especialización total y parcial;
- templates variádicos y fold expressions;
- concepts de C++20;
- iteradores, algoritmos y contenedores de la biblioteca estándar;
- separación entre estado, reglas, controlador e interfaz;
- pruebas unitarias y simulaciones reproducibles.

4. Escenario del juego

Una partida ocurre sobre una cuadrícula rectangular. El agente comienza en una celda inicial y debe llegar a la salida antes de quedarse sin energía o superar el límite de turnos.

Durante el recorrido puede encontrar:

- celdas libres;
- obstáculos que no pueden atravesarse;
- terrenos de costo elevado que consumen más energía;
- recursos que aumentan el puntaje;
- baterías que recuperan energía;
- trampas que reducen energía o puntaje;
- una salida que completa el escenario.

Una partida termina cuando ocurre una de estas condiciones:

- el agente alcanza la salida;
- su energía llega a cero;
- se alcanza el límite máximo de turnos.

El resultado debe indicar por lo menos si la partida fue completada, cantidad de turnos, energía restante, recursos recogidos y puntaje final.

5. Alcance funcional mínimo

5.1 Tablero

El tablero debe ser una clase genérica cuyas dimensiones formen parte del tipo:

```
template<typename Cell, std::size_t Rows, std::size_t Columns>
class Grid;
```

El escenario principal de demostración debe ser de **20 x 30**. Sus 600 celdas permiten rutas alternativas, zonas diferenciadas, callejones y decisiones de exploración más interesantes. La clase debe funcionar también con otras dimensiones conocidas en compilación; por ejemplo, las pruebas unitarias pueden utilizar cuadrículas de 3 x 4 para aislar reglas concretas.

La mayor dimensión no implica llenar el mapa de elementos. El escenario debe mantenerse legible, poseer por lo menos dos rutas parciales alternativas y tener una solución comprobada. No se requiere admitir dimensiones elegidas durante la ejecución.

Grid debe proporcionar:

- acceso validado a una posición;
- versiones const y no const del acceso;
- número de filas y columnas;
- iteradores para recorrer todas las celdas;
- consulta de posiciones válidas;
- una representación interna basada en `std::array`.

El tablero puede cargarse desde una colección de cadenas o construirse mediante una función auxiliar. No es necesario implementar un formato general de archivos.

5.2 Posición y dirección

Defina un tipo `Position` para representar fila y columna. Debe admitir las operaciones necesarias para:

- comparar posiciones;
- obtener una posición vecina;
- usar una posición como clave de un contenedor, si el diseño lo requiere;
- imprimirla o convertirla a texto para diagnóstico.

Las acciones básicas son discretas:

- mover arriba;
- mover abajo;
- mover a la izquierda;
- mover a la derecha;
- esperar, si se decide incluir esta acción.

No se requieren movimientos diagonales.

5.3 Agente

El agente posee como mínimo:

- posición actual;
- energía actual y máxima;
- puntaje acumulado;
- cantidad de recursos recogidos;
- estado activo o finalizado.

Deben preservarse estas invariantes:

- la posición del agente siempre pertenece al tablero;
- el agente nunca ocupa una celda bloqueada;
- la energía se mantiene entre cero y su máximo;
- un agente sin energía no puede ejecutar otra acción;
- un recurso recogido no puede recogerse nuevamente.

5.4 Movimiento

En cada turno el controlador selecciona una acción. El motor debe:

1. calcular la posición candidata;

2. rechazar movimientos fuera del tablero o hacia obstáculos;
3. actualizar la posición cuando el movimiento sea válido;
4. descontar el costo de energía correspondiente;
5. resolver la interacción con la celda de destino;
6. comprobar la condición de término.

`availableActions()` informa a los controladores qué movimientos producirían un desplazamiento válido, además de `wait`. No impide que un jugador humano intente otra dirección: `step` debe aceptar el intento y aplicar la regla de movimiento rechazado definida más adelante.

Mientras la partida esté activa, toda llamada válida a `step` consume exactamente un turno. Un movimiento hacia un muro o fuera del tablero no cambia la posición, pero aplica el costo configurado para un intento inválido y genera `MovementRejectedEvent`; se sugiere un costo de una unidad de energía. La acción `wait` tampoco cambia la posición: aplica su costo configurado, sugerido en una unidad de energía, y no activa la celda actual. El efecto de ambas acciones es obligatorio, aunque sus costos numéricos pueden variar si se documentan y prueban.

5.5 Tipos de terreno e interacciones

El escenario principal debe implementar los siete tipos de celda siguientes. Los valores numéricos de la tabla son sugeridos para el escenario de demostración, no constantes obligatorias. Cada grupo puede modificarlos si los declara en un lugar configurable, los explica en el `README.md` y adapta sus pruebas a los valores elegidos. La semántica descrita para cada celda sí es obligatoria.

Celda	Costo sugerido	Efecto sugerido después del movimiento	Frecuencia
Espacio libre	1 energía	Ninguno.	Cada entrada
Muro	1 energía por intento	El movimiento se rechaza y la posición no cambia.	Cada intento
Terreno de costo elevado	2 de energía	Ninguno adicional.	Cada entrada
Recurso	1 energía	Suma 10 puntos e incrementa en uno los recursos recogidos.	Solo la primera vez
Batería	1 energía	Recupera 3 de energía, sin superar la energía máxima.	Solo la primera vez
Trampa	1 energía	Resta 2 de energía y 1 punto adicional.	Cada entrada
Salida	1 energía	Completa la partida si queda por lo menos una unidad de energía.	No aplica

Perfiles de dificultad sugeridos

Se recomienda centralizar los valores variables en una estructura como `GameRules` y ofrecer tres configuraciones de fábrica. La interfaz puede recibir el nivel mediante una opción como `--difficulty easy, standard o hard`. Para la demostración evaluada se sugiere utilizar `standard`.

Parámetro	Easy	Standard	Hard
Energía inicial y máxima	80	60	40
Límite de turnos	240	180	140
Costo de espacio libre, recurso, batería, trampa o salida	1	1	1
Costo de terreno elevado	2	2	3
Costo de <code>wait</code> o intento inválido	1	1	1
Puntos por recurso	+15	+10	+8
Recarga de batería	+5	+3	+2
Penalización de energía por trampa	-1	-2	-3
Penalización de puntaje por trampa	0	-1	-2

Recomendación de diseño

Los perfiles deben ser datos, no ramas `if` dispersas por el motor. Por ejemplo, `rulesFor(Difficulty::standard)` puede devolver un `GameRules` con todos los costos y recompensas. `NavigationEnvironment` aplica las reglas que recibe y no necesita conocer el nombre del nivel.

Estos perfiles son referencias de balance, no valores obligatorios. Un grupo puede ajustarlos o añadir otro nivel siempre que conserve `standard` para la demostración común, registre los valores efectivos en el `README.md` y tenga pruebas que verifiquen la configuración seleccionada.

Los recursos y las baterías son consumibles. Después de activarlos deben marcarse como recogidos o consumidos, o reemplazarse por `Empty`; al volver a esa posición se comportan como un espacio libre. Las trampas no son consumibles y vuelven a aplicar su penalización en cada entrada. El puntaje puede ser negativo. Toda modificación de energía debe limitarse al intervalo entre cero y la energía máxima.

El orden obligatorio para resolver una acción es:

1. incrementar el turno;
2. si la acción es inválida, descontar el costo configurado, generar el evento de rechazo y pasar directamente a comprobar el término;
3. si la acción es `wait`, descontar su costo configurado sin activar ninguna celda y pasar a comprobar el término;
4. si el movimiento es válido, actualizar la posición y descontar el costo de entrada de la celda destino;
5. aplicar el efecto de la celda destino, incluso si el costo de entrada redujo temporalmente la energía a cero; de este modo, una batería todavía puede recargar al agente;
6. producir los eventos en el mismo orden en que ocurrieron los cambios;
7. comprobar las condiciones de término.

Si varias condiciones coinciden en el mismo turno, se aplica esta precedencia: `goalReached` cuando el agente está en la salida y conserva energía; después `noEnergy`; finalmente `turnLimit`. Por tanto, llegar a la salida con energía cero no constituye una victoria, mientras que llegar con energía positiva en el último turno sí la constituye.

Las celdas no deben imprimir directamente ni leer desde la consola. Deben producir un resultado o evento que describa la interacción realizada.

5.6 Controladores

El motor debe admitir controladores intercambiables. Como mínimo incluya:

- controlador humano, que recibe una decisión proporcionada por la interfaz;
- controlador aleatorio, que elige una acción legal;
- controlador heurístico simple, por ejemplo uno que reduzca la distancia Manhattan a la salida y evite obstáculos inmediatos.

No se requiere que la estrategia heurística encuentre siempre el camino óptimo. Las estrategias deben poder reemplazarse sin modificar el entorno.

5.7 Contratos públicos mínimos

Las siguientes interfaces constituyen una referencia precisa del contrato esperado. Se permite cambiar nombres o dividir responsabilidades de otra forma, siempre que se conserve el comportamiento, la separación de capas y la posibilidad de ejecutar las pruebas sin consola.

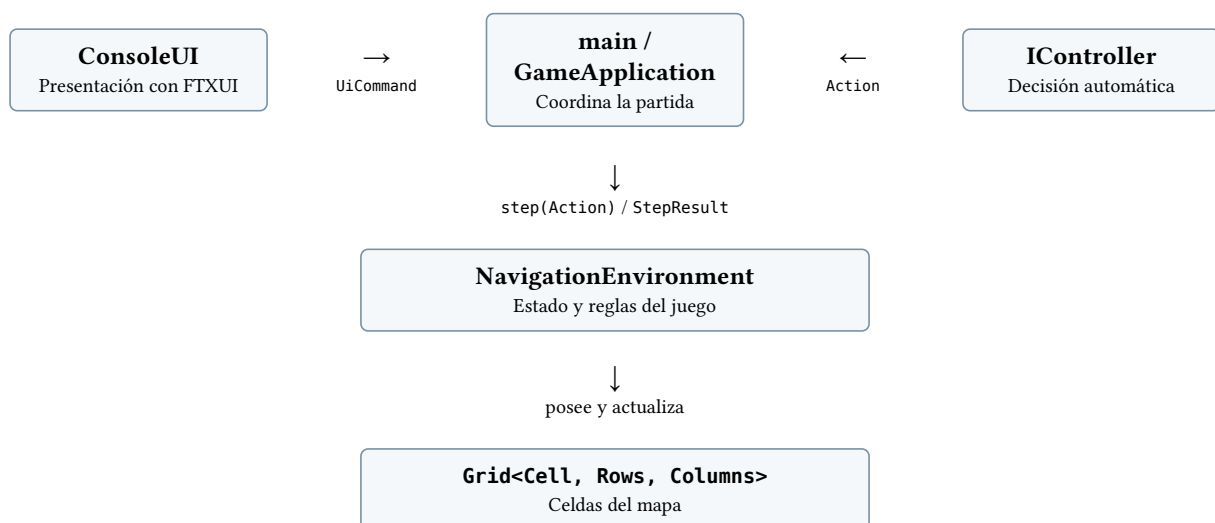


Figura 1: Colaboración entre las capas durante una partida.

Las flechas continuas representan llamadas o datos que circulan durante una partida; no representan herencia, navegabilidad de un diagrama UML ni mensajes de red. La única flecha discontinua muestra una relación de implementación.

Elemento	Responsabilidad	No debe hacer
ConsoleUI	Dibujar el tablero con FTXUI y convertir pulsaciones en UiCommand.	Aplicar energía, movimiento o modificar celdas.
main o GameApplication	Coordinar el ciclo: pedir una acción humana o automática, llamar a step y solicitar un nuevo renderizado.	Duplicar las reglas que pertenecen al entorno.
NavigationEnvironment	Mantener el estado de la partida, validar y ejecutar acciones, aplicar interacciones y producir resultados.	Leer el teclado, imprimir o depender de FTXUI.
Grid<Cell, Rows, Columns>	Almacenar las celdas, comprobar límites y permitir su recorrido mediante iteradores.	Conocer turnos, energía, controladores o reglas de victoria.
IController	Definir la interfaz común para que un controlador automático seleccione una acción.	Modificar directamente el entorno o realizar entrada/salida.
PolicyController<Policy>	Adaptar una política genérica válida a la interfaz virtual IController.	Decidir mediante typeid, dynamic_cast o condicionales por tipo.

Los nombres que aparecen sobre las flechas describen operaciones y valores del programa:

- Action es un valor como up, down, left, right o wait.
- UiCommand representa una acción humana, ayuda o salida.
- Observation es una copia inmutable de lo que el controlador puede observar.
- las acciones disponibles son las alternativas legales incluidas dentro de la observación;
- un evento describe algo que ya ocurrió, como movimiento, recolección o cambio de energía;
- StepResult reúne la nueva observación, los eventos producidos y la condición de término.

NavigationEnvironment es la única pieza que modifica el estado del juego. La UI y los controladores solo reciben copias o vistas const y proponen una acción. El nombre correcto de la interfaz es IController; no se utilizará un tipo distinto llamado IControl.

Tipos básicos

```
struct Position {
    std::size_t row{};
    std::size_t column{};

    friend bool operator==(const Position&, const Position&) = default;
};

enum class Action { up, down, left, right, wait };

std::optional<Position> neighbor(Position origin, Action action);
std::string toString(Position position);
```

neighbor no debe conocer el tamaño del tablero. Debe retornar std::nullopt cuando el cálculo intentaría producir una fila o columna negativa. La validación del límite superior corresponde a Grid.

Celdas

Una representación posible es un std::variant de tipos pequeños:

```
struct Empty {};
struct Wall {};
struct RoughTerrain { int energyCost{2}; };

template<typename Reward>
struct ResourceCell {
    Reward reward;
    bool collected{false};
};
```

```

struct Battery { int energy{3}; bool consumed{false}; };
struct Trap { int energyPenalty{2}; int scorePenalty{1}; };
struct Exit {};

using Cell = std::variant<
    Empty, Wall, RoughTerrain, ResourceCell<int>, Battery, Trap, Exit>;

```

Los estados collected y consumed permiten expresar que una interacción no se repite. También es válido reemplazar la celda por Empty después de consumirla, siempre que el cambio quede reflejado en el estado del entorno.

Cuadrícula

```

template<typename CellType, std::size_t Rows, std::size_t Columns>
class Grid {
public:
    using value_type = CellType;
    using iterator = /* iterador de la representación */;
    using const_iterator = /* iterador const de la representación */;

    static constexpr std::size_t rows() noexcept { return Rows; }
    static constexpr std::size_t columns() noexcept { return Columns; }

    [[nodiscard]] constexpr bool contains(Position position) const noexcept;

    CellType& at(Position position);
    const CellType& at(Position position) const;

    iterator begin() noexcept;
    iterator end() noexcept;
    const_iterator begin() const noexcept;
    const_iterator end() const noexcept;
    const_iterator cbegin() const noexcept;
    const_iterator cend() const noexcept;

private:
    std::array<CellType, Rows * Columns> cells_{};
};

```

at debe validar ambas coordenadas y lanzar std::out_of_range si la posición no existe. Los iteradores recorren en orden por filas: de izquierda a derecha y de arriba hacia abajo. Se debe impedir mediante static_assert o requires la instanciación con cero filas o cero columnas.

Estado observable y eventos

```

struct Observation {
    Position agent;
    Position goal;
    int energy{};
    int maximumEnergy{};
    int score{};
    std::size_t collectedResources{};
    std::size_t turn{};
    std::vector<Action> availableActions;
};

struct MovedEvent { Position from; Position to; int energyCost; };
struct MovementRejectedEvent { Position from; Action action; };
struct ResourceCollectedEvent { Position at; int points; };
struct EnergyChangedEvent { int previous; int current; };
struct TrapTriggeredEvent { Position at; };
struct GoalReachedEvent { Position at; };

using NavigationEvent = std::variant<
    MovedEvent,
    MovementRejectedEvent,
    ResourceCollectedEvent,
    EnergyChangedEvent,

```

```

    TrapTriggeredEvent,
    GoalReachedEvent>;

enum class EndReason { none, goalReached, noEnergy, turnLimit };

struct StepResult {
    Observation observation;
    std::vector<NavigationEvent> events;
    bool finished{false};
    EndReason reason{EndReason::none};
};

```

La observación no entrega referencias modificables al estado interno. Si el controlador necesita conocer el mapa completo, puede añadirse una vista const o una codificación inmutable del tablero. El motivo de término evita inferir el resultado examinando mensajes de texto.

Entorno

```

template<std::size_t Rows, std::size_t Columns>
class NavigationEnvironment {
public:
    using grid_type = Grid<Cell, Rows, Columns>;

    NavigationEnvironment(grid_type initialGrid,
                          Position start,
                          int initialEnergy,
                          std::size_t turnLimit);

    void reset(std::uint32_t seed);
    [[nodiscard]] Observation state() const;
    [[nodiscard]] std::vector<Action> availableActions() const;
    [[nodiscard]] bool isFinished() const noexcept;
    [[nodiscard]] StepResult step(Action action);
    [[nodiscard]] const grid_type& grid() const noexcept;
};

```

Precondiciones del constructor:

- start y la salida pertenecen al tablero;
- la posición inicial es transitable;
- existe exactamente una salida;
- la energía inicial y el límite de turnos son positivos.

Una precondición incumplida debe producir `std::invalid_argument`. `step` no lee ni imprime. Si se invoca después de terminar la partida, debe lanzar `std::logic_error`. En este proyecto, un movimiento hacia un muro o fuera del tablero **consume un turno y aplica el costo de energía configurado**, genera `MovementRejectedEvent` y no cambia la posición. Se sugiere un costo de una unidad; el grupo puede modificarlo si documenta y prueba el valor elegido.

Controladores y políticas

```

class IController {
public:
    virtual ~IController() = default;
    virtual Action selectAction(
        const Observation& observation,
        std::span<const Action> legalActions) = 0;
};

template<typename Policy>
concept NavigationPolicy = requires(
    Policy& policy,
    const Observation& observation,
    std::span<const Action> actions) {
    { policy.selectAction(observation, actions) } -> std::same_as<Action>;
};

template<NavigationPolicy Policy>
class PolicyController final : public IController {

```

```
public:
    explicit PolicyController(Policy policy);

    Action selectAction(
        const Observation& observation,
        std::span<const Action> legalActions) override;

private:
    Policy policy_;
};
```

Se deben proporcionar políticas aleatoria y heurística. La política aleatoria recibe o posee un generador con semilla controlable. Para una partida humana, la interfaz de consola obtiene una acción y la entrega directamente a `step`; no es necesario que `IController` lea desde `std::cin`.

5.8 Interfaz mínima de consola

La entrega debe incluir una interfaz jugable basada en FTXUI. Esta biblioteca C++ funciona en Windows, Linux y macOS, soporta UTF-8 y caracteres de ancho completo, y se integra con CMake. PowerShell y Command Prompt (`cmd.exe`) son shells: el mismo ejecutable funciona desde ambos cuando se utiliza un host de consola moderno.

Esta exigencia no incluye al sistema operativo MS-DOS original. FTXUI no lo soporta y una terminal DOS real tampoco ofrece una presentación portable de emojis. Si «DOS» significa `cmd.exe` en Windows, sí queda cubierto. PDCurses sería una alternativa para MS-DOS real, pero sin garantizar emojis y con una ruta de compilación diferente.

FTXUI debe limitarse al ejecutable y a la capa de presentación. Las clases `Grid`, `NavigationEnvironment`, eventos y controladores no deben incluir cabeceras de FTXUI. De ese modo, el motor continúa siendo portable y sus pruebas pueden ejecutarse sin abrir una interfaz interactiva.

La dependencia debe fijarse a una versión concreta en CMake. Por ejemplo:

```
include(FetchContent)
FetchContent_Declare(ftxui
    GIT_REPOSITORY https://github.com/ArthurSonzogni/FTXUI
    GIT_TAG v7.0.3
)
FetchContent_MakeAvailable(ftxui)

target_link_libraries(navigation_game PRIVATE ftxui::ftxui)
```

La interfaz traduce eventos del teclado y presenta únicamente el estado producido por el entorno:

```
struct QuitCommand {};
struct HelpCommand {};
using UiCommand = std::variant<Action, QuitCommand, HelpCommand>;

enum class RenderMode { emoji, ascii };

class ConsoleUI {
public:
    explicit ConsoleUI(RenderMode mode = RenderMode::emoji);

    std::optional<UiCommand> translate(const ftxui::Event& event) const;

    ftxui::Element render(
        const NavigationEnvironment<20, 30>& environment,
        std::span<const NavigationEvent> recentEvents) const;

    ftxui::Element help() const;
};
```

La lectura debe aceptar, como mínimo, W, A, S y D para movimiento, E para esperar, H para ayuda y Q para abandonar. Debe ignorar diferencias entre mayúsculas y minúsculas. Una entrada desconocida muestra un mensaje y vuelve a solicitar un comando sin ejecutar `step`.

No se debe mostrar un campo de texto ni esperar que el usuario presione Enter. FTXUI entrega cada pulsación como un `ftxui::Event: Event::Character('w')` cumple el papel de una lectura inmediata semejante a `getch()`. `CatchEvent` recibe y procesa estos eventos dentro de `ScreenInteractive::Loop`.

No se necesita una consulta equivalente a `kbhit()` para el ciclo normal: FTXUI espera eventos y redibuja cuando corresponde. Si una extensión futura necesita actualizar la simulación aunque no haya una tecla, puede utilizar `ftxui::Loop::RunOnce()` o publicar un `Event::Custom`; esto queda fuera del alcance mínimo.

La aplicación coordina el ciclo mediante `ftxui::ScreenInteractive`. La UI no modifica directamente al agente ni las celdas:

```
auto component = ftxui::Renderer([&] {
    return ui.render(environment, recentEvents);
});

component = ftxui::CatchEvent(component, [&](const ftxui::Event& event) {
    auto command = ui.translate(event);
    // visitar command; una Action se entrega a environment.step(...)
    return command.has_value();
});
```

Por ejemplo, `translate` puede reconocer tanto letras como flechas:

```
if (event == ftxui::Event::Character('w') ||
    event == ftxui::Event::ArrowUp) {
    return Action::up;
}
```

Convención visual

Emoji	ASCII alternativo	Significado
	@	agente
	.	espacio libre
	#	muro
	~	terreno de costo elevado
	R	recurso
	B	batería o recarga de energía
	T	trampa
	S	salida

El modo emoji es la presentación predeterminada y forma parte de la evaluación. El modo ASCII debe poder activarse mediante una opción como `--ascii` para ejecutar el programa en terminales que no representen correctamente Unicode. Los textos fuente y la salida se manejarán como UTF-8. No se debe suponer que `std::string::size()` corresponde al ancho visual de un emoji.

Ejemplo de la vista requerida para una cuadrícula de 20 x 30. Los muros del borde se conservan. Las coordenadas se dibujan fuera de la grilla, pero pegadas al borde sin columnas ni filas de separación. La esquina entre ambas reglas se presenta con , el mismo emoji utilizado para un espacio vacío. A su derecha, la regla horizontal comienza en  y repite  a  tres veces; debajo, la regla vertical también comienza en  y repite las unidades para las 20 filas.

En FTXUI la regla no debe construirse concatenando una sola cadena. Cada número y cada celda deben ser elementos independientes con un ancho exacto de dos columnas. Así, la coordenada y el emoji comparten la misma geometría:

```
using namespace ftxui;

Element coordinateCell(std::string label) {
    return text(std::move(label)) | size(WIDTH, EQUAL, 2);
}

const std::array<std::string, 10> keycapDigits{
    ", ", ", ", ",
    ", ", ", ", ""
```

```
};

Elements horizontalLabels;
for (std::size_t column = 0; column < 30; ++column) {
    horizontalLabels.push_back(
        coordinateCell(keycapDigits[column % 10]));
}

auto horizontalCoordinates = hbox({
    coordinateCell("□"), // intersección visual de los ejes
    hbox(std::move(horizontalLabels)),
});
```

Cada fila del tablero comienza con la coordenada vertical y continúa inmediatamente con sus 30 celdas. Tanto coordenadas como celdas deben aplicar `size(WIDTH, EQUAL, 2)`:

```
Elements renderedRow;
renderedRow.push_back(coordinateCell(keycapDigits[row % 10]));

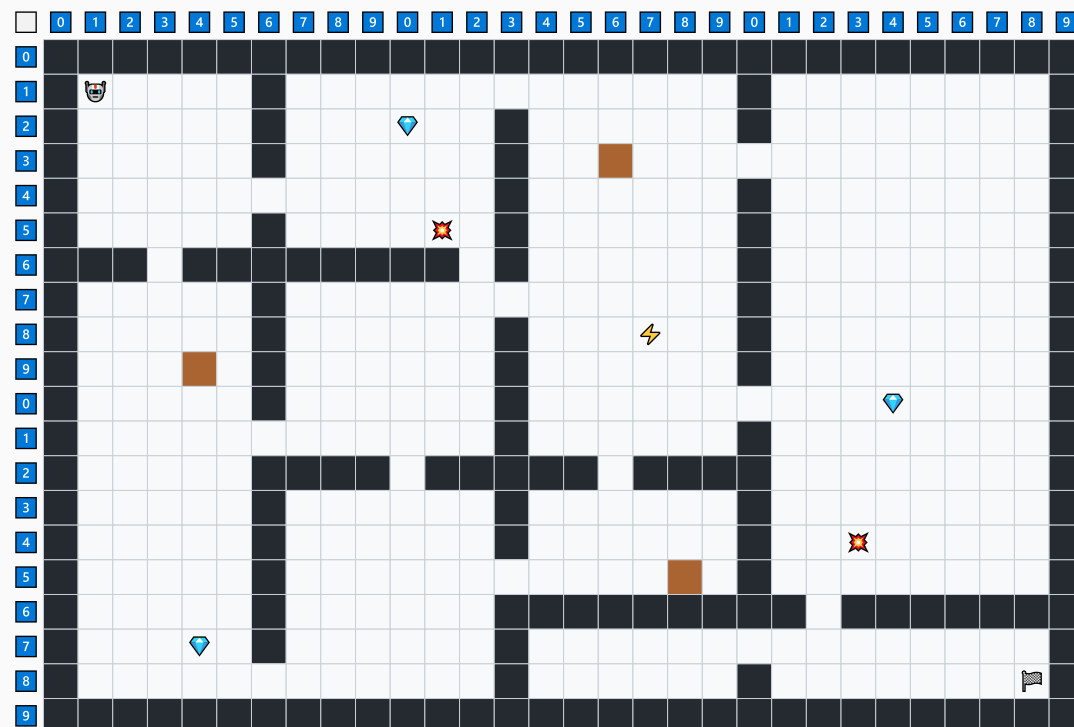
for (std::size_t column = 0; column < 30; ++column) {
    auto glyph = glyphFor(environment.grid().at({row, column}));
    renderedRow.push_back(
        text(glyph) | size(WIDTH, EQUAL, 2));
}

return hbox(std::move(renderedRow));
```

La fila de coordenadas y cada fila del tablero miden 62 columnas visuales: dos para la etiqueta vertical o el `□` de la esquina, más 60 para las 30 celdas. Los muros del borde continúan visibles y conservan el tipo `Wall` en el motor.

Los emojis numéricos son secuencias de varios puntos de código. Por esa razón no se debe medir su ancho con `std::string::size()`: FTXUI debe colocar cada uno en un elemento con `size(WIDTH, EQUAL, 2)`. El modo ASCII conserva dígitos normales como alternativa para una terminal incompatible.

Turno 27/180 | Energía 42/60 | Puntaje 20 | Recursos 2/3



💎 +10 en (10,24) | WASD mover · E esperar · H ayuda · Q salir

La primera línea funciona como barra de estado y debe aparecer siempre encima de la grilla. El pie combina el último evento con una ayuda breve; no existe un prompt Comando: ni un cursor de entrada. Para mantener la interfaz minimalista, H puede abrir temporalmente la ayuda completa.

El tablero, la energía, el turno, el puntaje, los recursos y el último resultado deben permanecer visibles. No se exige utilizar colores. La vista emoji está pensada para una terminal moderna de aproximadamente 80 columnas, suponiendo un ancho usual de dos columnas por emoji; si la terminal utiliza otra métrica, el usuario debe poder seleccionar el modo ASCII.

6. Requisitos de diseño y programación

6.1 Templates de funciones

Se deben implementar y utilizar por lo menos tres templates de función propios. Al menos dos deben recibir rangos mediante iteradores.

Posibles operaciones:

- contar celdas que cumplen una condición;
- buscar la primera celda de un tipo;
- seleccionar la mejor acción según una función de puntaje;
- transformar un rango de posiciones en acciones;
- calcular el costo total de una ruta;
- copiar posiciones transitables hacia otro contenedor.

Por lo menos uno de estos templates debe utilizarse con dos tipos de dato o dos tipos de contenedor diferentes.

No se debe duplicar un algoritmo únicamente porque un conjunto de posiciones se encuentre en un vector, list u otro contenedor.

6.2 Templates de clases

La clase principal `Grid<Cell, Rows, Columns>` debe utilizar:

- un parámetro de tipo para la celda;
- dos parámetros no-tipo para las dimensiones;
- almacenamiento contiguo;
- acceso mediante `operator()` o una operación equivalente;
- iteradores `const` y `no const`.

También se puede implementar una estructura de capacidad fija para el historial reciente:

```
template<typename T, std::size_t Capacity>
class FixedHistory;
```

El historial es opcional, pero puede ser útil para detectar ciclos o mostrar las últimas acciones.

6.3 Especialización

El proyecto debe utilizar:

- una especialización total;
- una especialización parcial.

Las especializaciones deben resolver una diferencia real del dominio y su resultado debe utilizarse en el programa.

Una alternativa es definir traits para las celdas:

```
template<typename Cell>
struct CellTraits {
    static constexpr bool traversable = true;
    static constexpr int energyCost = 1;
};

template<>
struct CellTraits<Wall> {
    static constexpr bool traversable = false;
    static constexpr int energyCost = 0;
};
```

```
template<typename Reward>
struct CellTraits<ResourceCell<Reward>> {
    static constexpr bool traversable = true;
    static constexpr int energyCost = 1;
};
```

La especialización total representa el caso concreto de un muro. La parcial reconoce la familia de recursos, independientemente de la representación de su recompensa.

Se puede proponer otro rasgo si la diferencia entre el caso general y los casos especializados queda justificada.

6.4 Templates variádicos

El proyecto debe usar un paquete variádico y por lo menos una fold expression.

Se recomienda representar los eventos mediante `std::variant`:

```
using NavigationEvent = std::variant<
    MovedEvent,
    MovementRejectedEvent,
    ResourceCollectedEvent,
    EnergyChangedEvent,
    GoalReachedEvent>;
```

Los eventos pueden procesarse mediante `std::visit` y la clase variádica `Overloaded`:

```
template<typename... Callables>
struct Overloaded : Callables... {
    using Callables::operator()...;
};

template<typename... Callables>
Overloaded(Callables...) -> Overloaded<Callables...>;
```

Utilice además una fold expression para una operación observable, como verificar varias condiciones de término, publicar varios eventos o combinar modificadores del costo de movimiento.

6.5 Concepts y polimorfismo dinámico

El proyecto debe combinar templates, concepts y una interfaz virtual.

Una alternativa es utilizar políticas de selección de acciones adaptadas a un controlador polimórfico:

```
template<typename Policy, typename State, typename ActionRange>
concept NavigationPolicyFor =
    requires(Policy& policy,
             const State& state,
             const ActionRange& actions) {
        { policy.selectAction(state, actions) } -> std::same_as<Action>;
};

template<typename State>
class Controller {
public:
    virtual ~Controller() = default;
    virtual Action selectAction(
        const State& state,
        const std::vector<Action>& actions) = 0;
};
```

Un adaptador genérico puede implementar `Controller<State>` para cualquier política que satisfaga `NavigationPolicyFor`. El concept verifica el contrato en compilación y la interfaz virtual permite cambiar el controlador en ejecución.

Se puede plantear otro diseño equivalente si se explica claramente:

- qué condición verifica el concept;
- qué clase es genérica;

- dónde ocurre el despacho dinámico;
- por qué no se utiliza un condicional según el tipo de controlador.

6.6 Biblioteca estándar

Se deben utilizar justificadamente varios elementos de la biblioteca estándar, entre ellos:

- `std::array` para el tablero de tamaño fijo;
- `std::vector`, `std::deque` o `std::list` para rutas, eventos o acciones;
- `std::variant` y `std::visit` para eventos o tipos de celda;
- `std::optional` para búsquedas que pueden fallar;
- `std::unique_ptr` para controladores polimórficos;
- algoritmos de `<algorithm>`;
- `<random>` con una semilla controlable;
- `std::unordered_map` o `std::map` cuando su uso esté justificado.

El README deberá explicar brevemente la elección de los contenedores principales.

FTXUI es la única dependencia externa requerida y se utiliza exclusivamente en la capa de presentación. Su uso no reemplaza ni cuenta como evidencia de los contenedores, algoritmos, templates o utilidades de la biblioteca estándar solicitados en este proyecto.

7. Preparación para el segundo proyecto

En este proyecto no se implementará reinforcement learning. Sin embargo, el motor debe funcionar como un entorno de simulación independiente del controlador.

La API exacta se definirá posteriormente, pero el diseño debe permitir operaciones equivalentes a:

```
environment.reset(seed);
auto state = environment.state();
auto actions = environment.availableActions();
auto result = environment.step(selectedAction);
bool finished = environment.isFinished();
```

Estado observable

El estado necesario para tomar una decisión debe poder obtenerse sin acceder a miembros privados ni analizar la salida de consola. Puede contener:

- posición del agente;
- energía actual;
- posición de la salida;
- tipo de las celdas visibles o del tablero completo;
- recursos restantes;
- cantidad de turnos;
- acciones legales.

El estado debe ser pequeño y preferentemente discreto. El escenario obligatorio de 20 x 30 produce 600 posiciones posibles y todavía es manejable para una primera versión tabular si el estado se limita a la posición y unas pocas variables discretizadas. Para la extensión de Q-learning se comenzará con una versión simplificada del escenario, sin codificar en el estado todas las combinaciones de energía, recursos y celdas consumidas, pues ese producto sí provocaría una explosión del espacio de estados.

Resultado de un paso

La ejecución de una acción debe producir información equivalente a:

```
struct StepResult {
    Observation observation;
    std::vector<NavigationEvent> events;
    bool finished;
};
```

No es obligatorio calcular una recompensa en este proyecto. Sin embargo, el resultado debe contener información suficiente para que posteriormente pueda asignarse, por ejemplo:

- una pequeña penalización por turno;
- una penalización por movimiento inválido;

- una recompensa por recoger recursos;
- una recompensa mayor por llegar a la salida;
- una penalización por agotar la energía.

El controlador no debe formar parte del estado del entorno. Debe ser posible usar el mismo entorno con un controlador humano, aleatorio, heurístico o de aprendizaje.

8. Pruebas mínimas

Se deben incluir pruebas para:

- acceso válido e inválido a la cuadrícula;
- bordes y esquinas;
- movimiento libre;
- movimiento contra un obstáculo;
- movimiento fuera del tablero;
- costos configurados para movimiento normal, terreno elevado, wait e intento inválido;
- recolección única de un recurso usando la recompensa configurada;
- batería consumible, recarga limitada por la energía máxima y posibilidad de recuperarse después de pagar el costo de entrada;
- activación repetida de una trampa y aplicación de ambas penalizaciones;
- carga correcta de los perfiles easy, standard y hard;
- precedencia entre llegada a la salida, energía agotada y límite de turnos;
- llegada a la salida;
- término por falta de energía y por límite de turnos;
- acciones disponibles junto a un muro, en una esquina y después del término;
- controlador aleatorio y controlador heurístico;
- procesamiento de cada tipo de evento;
- algoritmos genéricos con rangos vacíos y diferentes contenedores;
- reproducción de una simulación con la misma semilla.
- traducción de comandos válidos de consola, sin distinguir mayúsculas;
- rechazo de un comando de consola desconocido sin modificar el entorno;
- representación emoji y ASCII de todas las clases de celda;
- conservación de 20 filas y 30 celdas por fila en el renderizado.

Se debe documentar también una prueba negativa de compilación que muestre el diagnóstico generado cuando una política no satisface un concept.

La lógica del entorno debe poder probarse sin leer desde `std::cin` ni escribir en `std::cout`.

9. Restricciones

- Usar C++20.
- No usar variables globales mutables.
- No usar `new` o `delete` directamente.
- No almacenar objetos polimórficos por valor.
- No decidir comportamientos mediante nombres de tipos, `typeid` o secuencias de `dynamic_cast`.
- No duplicar algoritmos para distintos contenedores.
- Mantener visibles las definiciones de los templates en el punto de instanciación.
- Separar el entorno, los controladores y la interfaz de consola.
- Utilizar una semilla configurable para toda decisión aleatoria.
- No implementar todavía reinforcement learning, redes neuronales ni búsqueda avanzada.

Quedan fuera del alcance:

- interfaz gráfica;
- juego en red;
- niveles de dimensiones dinámicas;
- mapas mayores que los necesarios para la demostración;
- movimiento en tiempo real;
- enemigos móviles o múltiples agentes;
- visión parcial compleja;
- generación procedural compleja;
- A* u otros algoritmos avanzados de búsqueda;

- entrenamiento de un agente.

10. Organización y formato de entrega

10.1 Estructura mínima del repositorio

Cada grupo debe entregar un único repositorio con una organización reconocible. Se recomienda la siguiente estructura; pueden añadirse carpetas, pero no debe mezclarse el código fuente con archivos generados por la compilación.

```
proyecto_1/
+-- CMakeLists.txt
+-- README.md
+-- .gitignore
+-- include/
|   +-- circuit_escape/
|       +-- grid.hpp
|       +-- cells.hpp
|       +-- environment.hpp
|       +-- controllers.hpp
|       +-- game_rules.hpp
+-- src/
|   +-- environment.cpp
|   +-- controllers.cpp
+-- app/
|   +-- main.cpp
+-- tests/
|   +-- grid_test.cpp
|   +-- environment_test.cpp
|   +-- interactions_test.cpp
|   +-- controllers_test.cpp
+-- assets/
|   +-- maps/
|       +-- scenario_01.txt
|       +-- scenario_02.txt
+-- docs/
|   +-- design.md
|   +-- contributions.md
```

Las definiciones de templates deben permanecer en cabeceras o en archivos .hpp incluidos desde ellas. app/main.cpp contiene la aplicación ejecutable; la lógica reutilizable permanece en include y src. La carpeta build, los ejecutables, archivos temporales y configuraciones del editor deben excluirse mediante .gitignore.

10.2 Contenido obligatorio

El repositorio debe incluir:

- código fuente y CMakeLists.txt capaz de construir el proyecto desde una carpeta limpia;
- pruebas automáticas ejecutables mediante CMake y CTest, o mediante el comando acordado en clase;
- programa de demostración en consola;
- por lo menos dos escenarios de 20 x 30 con solución alcanzable;
- perfiles sugeridos easy, standard y hard, o una justificación documentada de cualquier ajuste realizado;
- simulación automática reproducible con una semilla fija;
- README.md con integrantes, requisitos, compilación, pruebas, ejecución, controles, perfiles de dificultad y ejemplos;
- docs/design.md con las decisiones de diseño y la ubicación donde se aplica cada tema obligatorio;
- docs/contributions.md con el resumen verificable de las responsabilidades y contribuciones de cada integrante;
- instrucciones de ejecución para PowerShell o Command Prompt, Linux y macOS.

10.3 Entrega mediante GitHub

- Se utilizará un repositorio GitHub por grupo. El nombre sugerido es cs2013-2026-2-proyecto-1-grupo-NN.
- El repositorio debe ser privado durante el desarrollo y dar acceso al docente o a la organización del curso. Si se utiliza GitHub Classroom, prevalece el repositorio creado por la invitación del curso.
- La rama predeterminada será main y debe compilar, ejecutar las pruebas y conservar una demostración funcional.
- La entrega se identifica con el tag anotado proyecto-1-entrega. El grupo debe registrar en el medio indicado por el curso la URL del repositorio y el hash completo del commit etiquetado.

- Solo se evaluará el contenido alcanzable desde ese tag. Los commits posteriores pueden conservarse, pero no modifican la entrega registrada.
- No se aceptará como sustituto un archivo ZIP, una carpeta compartida o código enviado por correo, salvo indicación expresa del docente.

Antes de crear el tag, el grupo debe clonar el repositorio en una carpeta nueva, seguir únicamente las instrucciones del README .md, compilar, ejecutar las pruebas y verificar la demostración.

10.4 Flujo de branches y pull requests

Se recomienda un flujo basado en main y ramas de vida corta. No se requiere una rama permanente develop.

- Cada tarea comienza desde una versión actualizada de main.
- Use nombres descriptivos como feature/grid-iterators, feature/battery-rules, fix/energy-limit, test/trap-events o docs/readme-build.
- Una rama debe resolver una tarea concreta. Evite ramas personales que acumulen trabajo no relacionado durante varias semanas.
- Publique un pull request pequeño, describa el cambio, indique cómo probarlo y vincúlelo con el issue correspondiente.
- Se recomienda por lo menos una revisión de otro integrante antes de integrar. El autor no debe aprobar su propio pull request.
- No integre si la compilación o las pruebas fallan. Se recomienda activar una acción de integración continua, aunque no es obligatoria.
- Para reducir conflictos, actualice la rama con main antes de integrarla y coordine previamente los cambios sobre interfaces compartidas.
- Se recomienda Squash and merge para conservar un commit principal por tarea. Después de integrar, elimine la rama remota y local.
- No haga push directo a main, salvo una corrección excepcional acordada por todo el grupo y registrada en el repositorio.

Un ciclo de trabajo típico es:

```
git switch main
git pull --ff-only
git switch -c feature/battery-rules
# implementar, probar y crear commits enfocados
git push -u origin feature/battery-rules
# abrir pull request, revisar, integrar y eliminar la rama
```

10.5 Formación del grupo y división del trabajo

El proyecto es grupal y admite como máximo cinco integrantes. Los nombres, códigos y usuarios de GitHub deben figurar en el README .md. Se recomienda distribuir responsabilidades iniciales, pero ninguna área debe depender de una sola persona durante todo el proyecto.

Área sugerida	Responsabilidades principales
Integración	CMake, contratos compartidos, seguimiento de issues, integración de pull requests y versión demostrable.
Tablero y dominio	Position, Grid, celdas, carga de escenarios e invariantes.
Reglas y pruebas	GameRules, energía, interacciones, eventos y pruebas de regresión.
Controladores	Controladores humano, aleatorio y heurístico; concepts, polimorfismo y reproducibilidad.
Interfaz y documentación	FTXUI, modo ASCII, experiencia de uso, README .md y demostración multiplataforma.

Las áreas son una guía y pueden combinarse cuando el grupo tenga menos de cinco integrantes. Se aconseja trabajar mediante pequeñas integraciones verticales: una funcionalidad incluye su contrato, implementación, pruebas y documentación. Evite dividir el proyecto únicamente por archivos o asignar a una persona solo la documentación. Cada integrante debe contribuir código o pruebas, revisar al menos un pull request ajeno y ser capaz de explicar las decisiones principales.

Consejo de coordinación

Definan primero los contratos compartidos y registren las tareas como issues. Para una modificación que afecte a

Cell, Observation, StepResult o GameRules, acuerden la interfaz antes de abrir varias ramas dependientes. Una reunión breve de integración dos veces por semana suele ser más eficaz que resolver una gran cantidad de conflictos al final.

11. Plan de trabajo sugerido

Semana 1: tablero y estado

- creación del repositorio, README.md, estructura de carpetas e issues;
- Position, acciones y tipos de celda;
- Grid<Cell, Rows, Columns>;
- agente e invariantes;
- pruebas de límites y movimiento.

Semana 2: entorno

- definición de GameRules y perfiles de dificultad;
- resolución de acciones;
- energía e interacciones;
- condiciones de término;
- simulación sin interfaz de consola.

Semana 3: extensibilidad

- controladores aleatorio y heurístico;
- interfaz polimórfica y adaptador genérico;
- concepts;
- eventos, variant y templates variádicos.

Semana 4: integración

- interfaz de consola;
- escenarios de demostración;
- simulación reproducible;
- pruebas completas y documentación;
- ensayo de entrega desde un clon limpio y creación del tag final.

12. Posibles ventajas para la segunda etapa

Esta alternativa ofrece un espacio de decisiones especialmente sencillo:

- pocas acciones discretas;
- estados pequeños;
- episodios cortos;
- recompensas fáciles de interpretar;
- simulaciones rápidas;
- comportamiento reproducible.

Estas características facilitan una introducción posterior a Q-learning tabular. La ampliación podría comenzar con una cuadrícula sin recursos ni energía y usar únicamente posición, movimiento, obstáculos y salida. Después se podrían agregar gradualmente costos de terreno y recursos.

13. Riesgos de alcance

Para terminar en cuatro semanas se debe evitar:

- convertir el proyecto en un editor general de mapas;
- implementar un sistema complejo de archivos;
- agregar enemigos que también se desplacen;
- generar mapas aleatorios garantizando conectividad;
- implementar varios algoritmos de búsqueda;
- permitir tamaños de tablero arbitrarios en ejecución;
- diseñar una interfaz gráfica.

El alcance mínimo debe poder demostrarse con dos escenarios de 20 x 30 definidos directamente en el programa o cargados desde una representación textual simple.

14. Decisiones pendientes

Antes de publicar el enunciado definitivo todavía se debe precisar:

- framework de pruebas permitido;
- cantidad mínima definitiva de pruebas;
- tipos exactos de celda requeridos;
- si el tablero completo será observable en la segunda etapa;
- si `Grid<Cell, Rows, Columns>` será obligatorio con esa firma;
- qué especializaciones se aceptarán como evidencia válida;
- si los eventos mediante `variant` serán obligatorios;
- formato de la demostración final;
- criterios y pesos de la rúbrica;
- política sobre herramientas de IA generativa.